

在 VR 中用模态共振器合成物理接触声音

Synthesizing Physical Contact Sound in VR with Modal Resonators

一个把物理接触事件实时变成声音的模态合成器。collision / friction / material params → DSP → audio。

原理

① 物体的声音

每个物体都有自己最容易振动的频率。敲它，它就用这些频率响起来。这些频率叫**模态 (modes)**，由物体的形状、密度、材料、边界条件等物理参数共同决定。

一个物体的声音 = 它所有模态的叠加。

② 一个模态

描述一个模态只需要三个数字：

- 频率多高 (frequency)
- 起始多大声 (amplitude)
- 衰减多快 (decay)

有了这三个数字，你就可以计算出这个模态的声音本身。

③ 16 个模态

一个物体不止一个模态，而是几十、上百个模态一起响。只要计算出足够多的模态一起发出声音，听起来就好像是真实的声音一样。

但人耳只听得最强的一批，所以这个引擎用 16 个模态描述每种材质，已经能稳定还原一种材质的音色特征。

④ 和谐与不和谐

弦的模态频率是整数倍——听起来“和谐”，有清楚的音高。钟、板、玻璃杯等不规则物体则不是——它们的模态自由分布，听起来“不和谐”，更接近噪声。

在这个合成器里，用一个参数 **inharmonic** (不和谐度) 控制模态分布，从和谐滑到不和谐。

游戏开发集成

⑭ 声音池 (Voice Pool)

游戏启动时，DSP 端一次性预分配 64 个 voice——每个 voice 自己独立的 mode bank、状态、输出流。每次 collision event 从池里借一个空闲 voice 给这次激发用；voice 静默后还回池里。runtime 不分配内存——Quest 3 移动 CPU 受不起。

⑮ 物理抓取

Unity 提供的 XR Interaction Toolkit 抓起东西后它就“飘”在你手上一撞墙穿模、前后移动不自然。将其修改为了抓住的 cube 仍然是物理刚体。撞击有真实反作用，扔投靠手柄的运动学。

横向投掷自带“杠杆放大”，手腕的一个小转动，在远端被抓住的物体那里就被放大成大幅度的线速度。但前后投掷没有这层放大——只能靠手柄自己往前/往后平移，速度天然慢得多。

解法：松手那一刻，沿手柄前向额外补一点速度，跟抓握时手到物体的距离成正比。前后投掷的感觉就跟同距离的横向投掷一样有力。

严格说不是真物理，但符合玩家直觉。

⑯ 触觉跟着声音走

控制器震动实时跟随声音的音量包络——手感觉到的、耳朵听到的，在同一条时间轴上。

价值

⑤ 录音的问题

想录“木头碰金属”？录一遍。“金属碰塑料”？再录一遍。 $X \times Y \times Z$ 录制数量指数级增加。

参数化合成器只用 7 个数字描述每种材质：

1. freq —— 基频
2. decay —— 衰减时间
3. hf_damping —— 高频阻尼
4. inharmonic —— 不和谐度
5. tonality —— 音调度
6. hardness —— 硬度
7. roughness —— 粗糙度

⑥ 能算就能变化

录音录什么播什么，不会变。算出来的声音可以一边响一边变——拖一下材质滑块，正在响的声音 mid-flight 跟着变 timbre。

合成器在 audio thread 实时运行——Unity C# 脚本任何时候都能写入 DSP 参数 (如 hardness、tonality)，声音输出立即跟着变，无需重启或缓存切换。三角混合器 (Box 12) 即时演示

物理驱动

⑦ 物理事件驱动声音

Unity 物理引擎在每次碰撞给出：冲量、接触点、法线、PhysicsMaterial。

先做法向 / 切向分解：法向 → impact 路径 (撞击)，切向 → friction 路径 (摩擦)——同一物理事件、两条独立激发器。详见背面 S3 / S4

⑧ 两个物体一起发声

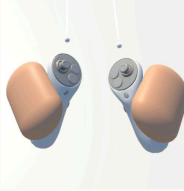
金属球落木地板，声音是**两者耦合**。一个 voice 同时持有两侧材质，互相吸收能量。

一个 voice 内部并行两侧 mode bank (A、B)。接触保持时，A 的 decay 被 B 阻尼缩短 (钢碰皮肤，1.8s → 0.45s)——这就是 cross-damping。详见背面 S6 / S7

Demo —— 你可以试试

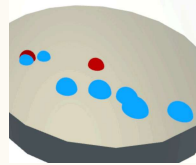
⑨ 物理手部

戴上 Quest 抬起手——控制器上出现的一双手，是**物理刚体**，不是视觉 mesh。能推、能撞、能抓——跟虚拟世界里的所有物体真实交互。所有 demo 共同的入口。



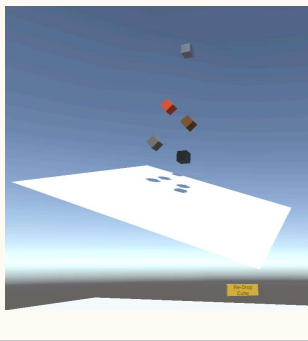
⑩ 视觉标记

撞击时接触点画**红点** (大小对应冲击力)；摩擦时沿运动轨迹每 tick 画一颗**蓝点** (大小对应当下滑动速度)。声、触、视三轴在同一个物理事件上同步——看到的、听到的、手感觉到的是同一件事。



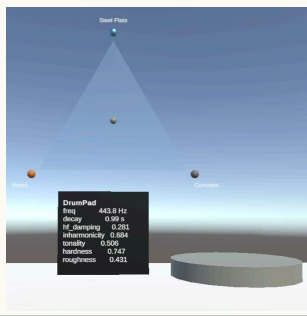
⑪ 方块掉落斜坡

点击按钮，聆听方块掉落后自然滚动 / 滑动的声音。



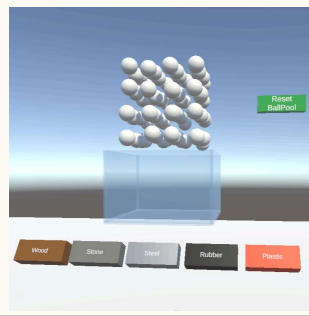
⑫ 三角混合器

三个顶点是三种材质。拖一下中间的小球，drumpad 跟着变音色。



⑬ 球池

伸手进去搅一搅，听听自然涌现出来的颗粒感 (granularity)。还可以变换材质。



李墨非 Mofei Li

<https://mofei.me/> • mofei_li@qq.com • GitHub: github.com/MofeiLi1



空间音频

⑰ Steam Audio + HRTF

每个 voice 的空间位置直接进 Steam Audio，通过一个通用 HRTF (头部相关传输函数) 渲染。

HRTF: 人耳判断声音方位除了左右耳音量差和时间差之外，还会受到耳朵轮廓和头部躯干的影响，使得不同角度的声音抵达耳膜时会有细微的 EQ 和 delay 差别。把带空间信息的音频输入 HRTF 输出一个模拟人耳真实听感的立体声——即使用户实际听到的只是双声道，也能分辨前后上下。

声音系统架构

⑱ Unity ↔ DSP 集成

从 Faust 源码到耳机里的声音，一共 5 层——每一层一个 C# / C++ 文件，清晰单职责。DSP 内部 signal flow 详见背面

1. DSP 层——modal-resonator.dsp (Faust ~300 行) → 编译成 C++ → sound_engine.cpp 包成 SE_* 原生函数 → 跨编译成 Win DLL / Mac bundle / Android.so
2. P/Invoke 桥——NativeDSP.cs 用 [DllImport] 暴露 SE_Init / AllocVoice / SetMaterialPair / TriggerImpact / UpdateContact / Compute
3. Voice Pool——VoicePool.cs 是 Singleton。Awake() 调 SE_Init(sr, 64) 预分配 native voice + 创建 64 个 SpatialVoice GameObject
4. 游戏逻辑——CollisionSoundManager.cs 在 OnCollisionEnter / Stay 提物理量 + 调 SE_TriggerImpact / UpdateContact
5. 音频输出——SpatialVoice 持 AudioSource (spatialize on)，audio thread 拉 PCM 调 SE_Compute → 管线 → Steam Audio HRTF → 耳机

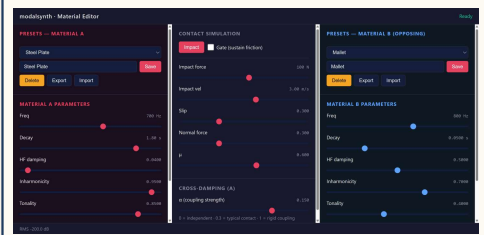
在线调音

⑲ WebUI 调音工具

浏览器里直接拖滑块调 7 个材质参数，实时听效果——不需要 VR 头显也能体验参数化合成。

和 VR demo 用的是同一份 DSP 源码——Web、Unity、Quest 三个目标都从这一个文件编译出来。

mofei.me



法向 / 切向分解:法向分量驱动 **impact** 路径(撞击瞬间), 切向分量驱动 **friction** 路径(持续摩擦)— 这是后面 S3 / S4 两条激发器路径分开的物理基础。

这两个量贯穿后面 **impact / friction** 两个激发器, 决定接触面的**接触动力学** — 频谱亮度、asperity 噪声深度。

撞击是 **Hertzian 接触应力波**(弹性接触理论)— 应力波从接触点扩散, 接触刚度决定波上沿陡度 → 决定频谱亮度。
Steel-Steel:cutoff $\approx$ 8500 Hz(尖锐 click)。Steel-Rubber: $\approx$ 2.6 kHz(几乎只剩基模, thud)。

整个 DSP 只用一个全局 **master_gain**, **impact** 和 **friction** 共享同一系数 — 相对响度只能由物理参数决定, 不允许 **impact** 比 **friction** 大声多少"这种 perceptual fudge。

Cross-damping 只改 **decay**, 不动 **damping_exp**(高频 rolloff 形状) — 后者是材料 A 自己的特性, 不该因接触 B 就变。

三个独立维度控制 mode bank 的"音色坐标":
• **inharmonicity** 控 mode 频率分布(harmonic 整数倍 → bar/plate 压缩)
• **tonality** 控每个 mode 共振峰锐度(清楚音色 vs 涂抹噪声)
• **hf_damping** 控高频比基频衰减更快多少(metallic ring vs 短闷响)

diffuse 纹理是每材质独立的高频沙沙感, 补充 modal bank 没覆盖的随机性。低 **tonality** 增强纹理权重 — Skin 摩擦比 Steel 更"沙"就来自这一层。

S1 Unity → DSP 输入

每次 collision, Unity 物理引擎产出几个量, **CollisionSoundManager** 提取后送进 DSP 的 slider:
One-shot(OnCollisionEnter): **impact_force** (冲量 ÷ dt=力)、**impact_vel** (相对速度)。
Per-tick(FixedUpdate): **gate** (0/1 接触)、**tangent_vel** (切向摩擦速度)、**normal_force** (法向力)、**friction_coeff** (μ , 从 **PhysicsMaterial**)。
加上 **14 个材质参数**(7×2 sides)从两侧 **SoundMaterial** 拿过来。

```
tangentVel = relVel - (relVel · n) · n
```

S2 接触面 Pair 属性

两侧材质的 **hardness / roughness** 不是简单平均, 而是各自有物理对应的合成。谐波平均让**较软的一侧主导接触刚度**;RSS(root-sum-square, 独立量方差相加)让两个独立粗糙表面的 **asperity** 高度分布相加。

```
combined_hardness = 2 · HA · HB / (HA + HB)  
combined_roughness = √(RA2 + RB2)
```

S3 Impact 激发器

每次撞击 trigger 一次:
• **env(AR)**:attack=0.5ms, release=1~21ms(由 combined_hardness 决定 — 软接触脉冲长)
• **amp** = √(**F · v**):**F · v** 是机械功率;开根让 **amp** ∝ √(power)。impact 和 friction **数学同构** (power-equivalent law)
• **cutoff**:500 + comb_hard · 8000 · T_{AB} + √v · 1200。硬接触亮、软接触暗;阻抗 mismatch 进一步把高频拉低

```
exciter_impact = noise · env(AR)  
· √(F · v) → lowpass(cutoff_imp)
```

S4 Friction 激发器

接触保持时, 每 tick 计算:
• **slip** = **tangent_vel**(asperity 微观凸起的碰撞频率)
• √(**N · μ**) = 单次 asperity 碰撞的峰值力
• η = 0.03 + roughness · 0.2(声辐射效率, 3-23%)
• **am_mod** 模拟 asperity 离散碰撞的随机时间结构 — **低速颗粒、高速嘶嘶**

```
cont_amp = slip · √(N · μ) · η · gate  
am_mod = 1 - depth · noise  
· lp(40+slip·3000)  
exciter_fric = noise · cont_amp  
· am_mod → lp(cont_cutoff)
```

微观粗糙表面之间 asperity 互相碰撞产生 **broadband 噪声**。粗糙度决定噪声谱形状, 法向力决定整体响度, 切向速度决定颗粒/嘶嘶节律。

S5 统一激发器 · fan-out

把 **impact** 和 **friction** 加起来, 乘 **master_gain**。
接下来同一份 **exciter** 信号 fan-out 给 A 侧和 B 侧 mode bank。

```
exciter = (exciter_impact  
+ exciter_friction) · 0.012
```

exciter ———→ Side A
 └→ Side B

S6 Cross-damping

接触保持时, A 侧的有效 **damping** 被 B 侧 **damping** 加上去, 反之亦然。能量泄漏给 B 后被 B 的 viscoelastic damping(粘弹性阻尼)耗散 → A 的 **decay** 缩短。cd_alpha 控制耦合强度, **demo** 中实际设为 **0.15**(DSP 默认 0.3)。

Steel + Steel	1.8s → 1.57s(匹配, 弱耦合)
Steel + Skin	1.8s → 0.45s (强耦合)
Rubber + Steel	几乎独立(阻抗 mismatch 极大)

```
hf_damp_eff_A = hf_damp_A  
+ s_gate · cd_alpha · hf_damp_B  
decay_eff_A = decay_A  
· (hf_damp_A / hf_damp_eff_A)
```

S7 Mode Bank A

每侧 16 个并行 **resonbp** (resonant bandpass)把 **exciter** 信号转成 modal 振动:

```
ratio_exp = 1 + 2.3 · B - 2.6 · B2  
damping_exp = hf_damping · 3  
Fi = freq · (i+1)ratio_exp  
T60i = decay_eff  
· (freq/Fi)damping_exp  
Gi = 1 / √(i+1)
```

S7 Mode Bank B

B 侧用材质 B 的参数(**freq_B**、**decay_eff_B**、**hf_damping_B**、**inharmonicity_B**、**tonality_B**)。同样的公式作用于不同 partials。
16 modes × 2 sides = 32 modes 一起响, 共享同一份 **exciter** 信号。

dual-bank 一个 voice + 一个 **spatializer** 的取舍:Quest 上 Steam Audio HRTF 是真瓶颈(每多一个 voice 多一个 HRTF kernel, 单价 100×), 但 DSP 内部多 modal bank 几乎免费(~16 IIR 状态)。把 **impact + friction + A + B** 都塞进同一份 **spatializer**, 比 dual-source 节省 **50% spatializer cost**。

S8 Diffuse 纹理 + 输出

每侧除了 16 个 modal, 再叠 4 个 **inharmonic** 高频纹理 mode(频率比 **1.7 / 3.3 / 5.7 / 9.1**), 按 **(1-tonality) · 0.5** 加权。
最终输出整个 voice **一份 mono 信号** → Unity 的 Steam Audio → HRTF 空间化成 3D 立体声(见**正面 Box 17**)。

```
side_A = modal_bank_A + diffuse_bank_A  
side_B = modal_bank_B + diffuse_bank_B  
output = (side_A + side_B)  
→ tanh(软限幅) → 复制成 stereo
```

符号
F: 冲击力
N: 法向力
 η : 声辐射效率
T_{AB}: 阻抗匹配
cd_a: 跨阻尼耦合
F_i/T60_i/G_i: mode i 频率/衰减/幅度
v: 相对速度
μ: 摩擦系数
gate: 接触开/关
B: 非谐性
master_gain: 全局增益

Pipeline 总览

